THE Forward-Forward Algorithm: Some Preliminary Investigations

이미지 처리팀

김병현, 김현진, 류채은, 안종식, 이주영, 최승준, 현청천(발표자)

목차

Part 1,

Abstract

Author

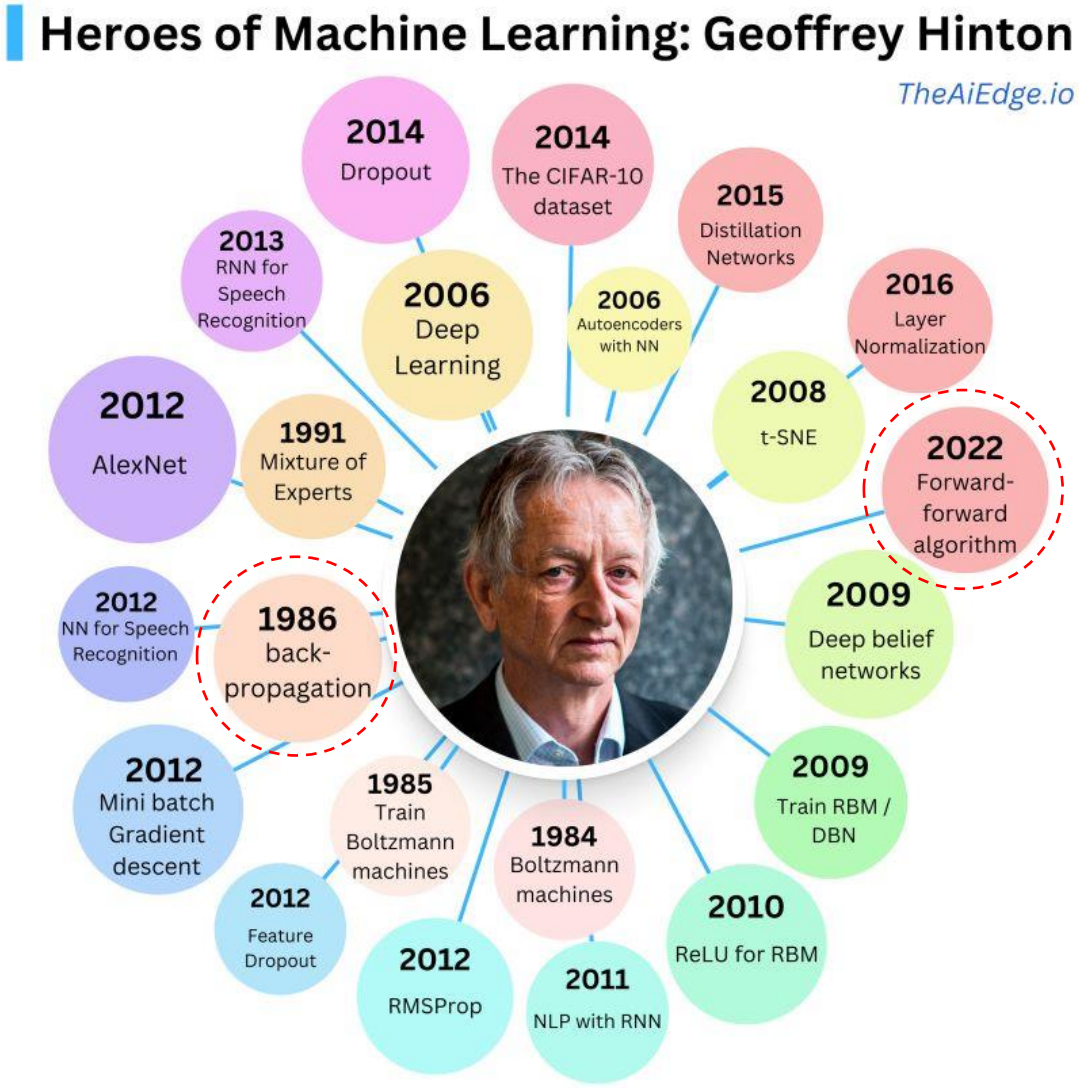
Geoffrey Hinton

팔로우 중

Emeritus Prof. Comp Sci, U.Toronto & Engineering Fellow, Google
cs.toronto.edu의 이메일 확인됨 - 홈페이지

machine learning psychology artificial intelligence cognitive science computer science

제목	인용	연도
Imagenet classification with deep convolutional neural networks A Krizhevsky, I Sutskever, GE Hinton Communications of the ACM 60 (6), 84-90	128930	2017
Deep learning Y LeCun, Y Bengio, G Hinton Nature 521 (7553), 436-44	61764	2015
Dropout: a simple way to prevent neural networks from overfitting N Srivastava, G Hinton, A Krizhevsky, I Sutskever, R Salakhutdinov The journal of machine learning research 15 (1), 1929-1958	41568	2014
Visualizing data using t-SNE L van der Maaten, G Hinton Journal of Machine Learning Research 9 (Nov), 2579-2605	34417	2008
Learning representations by back-propagating errors DE Rumelhart, GE Hinton, RJ Williams Nature 323 (6088), 533-536	31923	1986
Learning internal representations by error-propagation DE Rumelhart, GE Hinton, RJ Williams Parallel Distributed Processing: Explorations in the Microstructure of ...	30589	1986
Schemata and sequential thought processes in PDP models. D Rumelhart, P Smolenksy, J McClelland, G Hinton Parallel distributed processing: Explorations in the microstructure of ...	28017 *	1986
Learning multiple layers of features from tiny images A Krizhevsky, G Hinton	21475	2009
Rectified linear units improve restricted boltzmann machines V Nair, GE Hinton Proceedings of the 27th international conference on machine learning (ICML ...	20837	2010
Reducing the dimensionality of data with neural networks GE Hinton, RR Salakhutdinov Science 313 (5786), 504-507	19769	2006

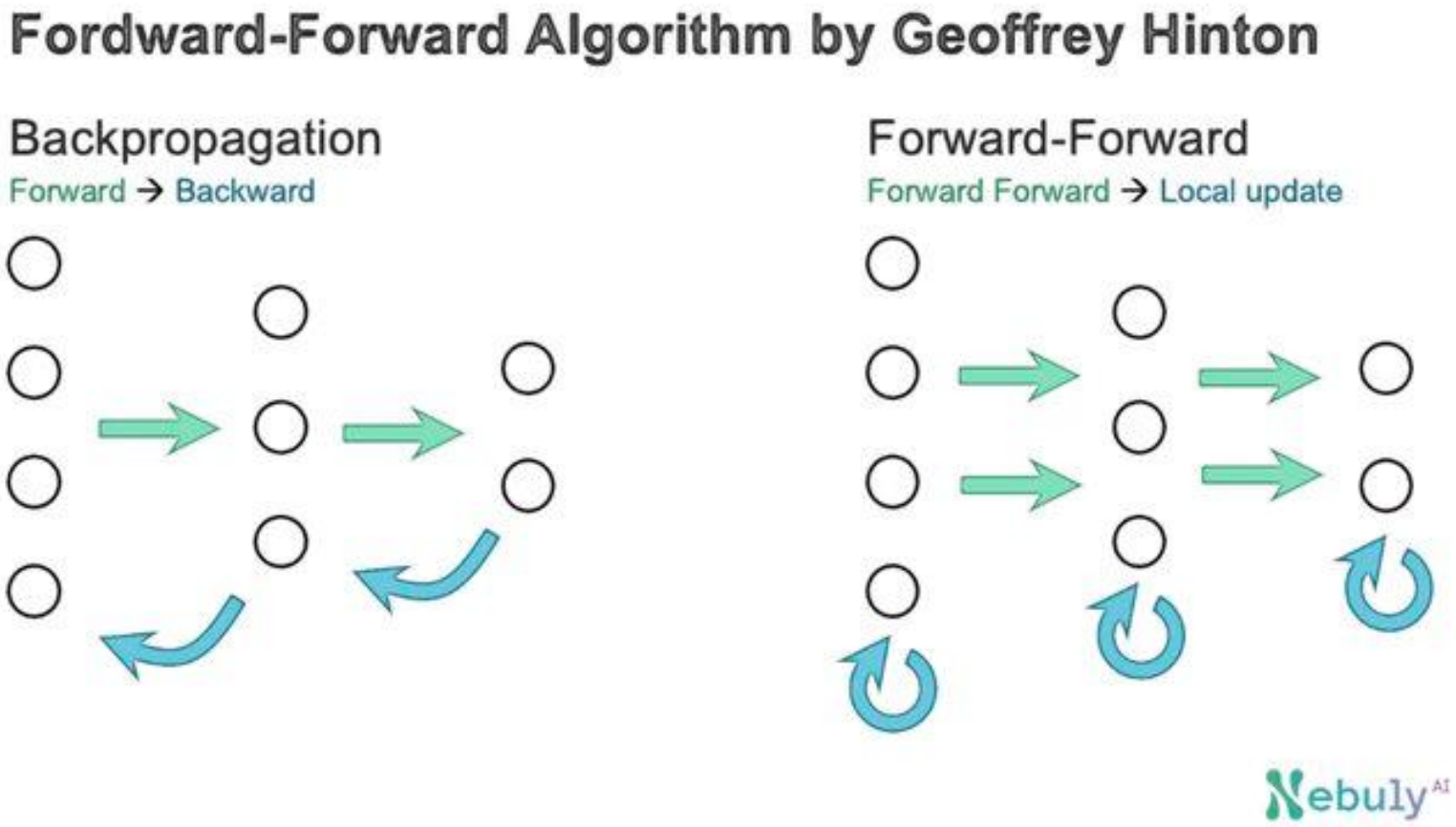


https://www.youtube.com/watch?v=l9RWTMNnvi4

Abstract

- The aim of this paper is to introduce **a new learning procedure for neural networks** and to demonstrate that it **works well enough on a few small problems** to be worth further investigation.
- The Forward-Forward algorithm **replaces the forward and backward passes of backpropagation by two forward passes**, one with **positive (i.e. real) data** and the other with **negative data** which could be generated by the network itself.
- **Each layer has its own objective function** which is simply to have **high goodness for positive data** and low **goodness for negative data**.
- The **sum of the squared activities in a layer can be used as the goodness** but there are **many other possibilities**, including minus the sum of the squared activities.

Abstract


Fordward-Forward Algorithm by Geoffrey Hinton
Backpropagation
Forward → Backward
Forward-Forward
Forward Forward → Local update
Nebuly AI

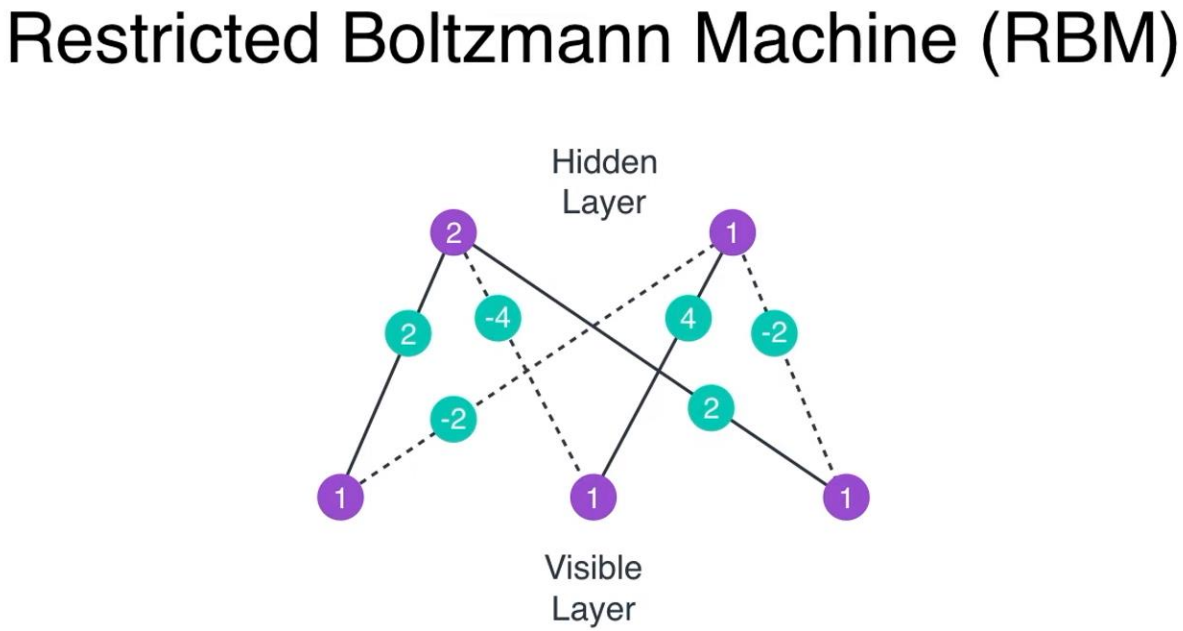
Part 2,

How FF relates to other contrastive learning techniques

Part 2,

How FF relates to other contrastive learning techniques

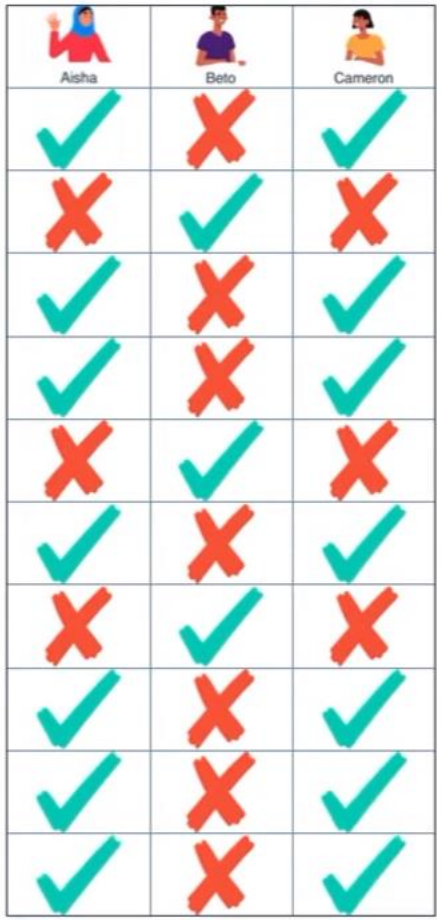
- **Relationship to Boltzmann Machines**
  - In the early **1980s there were two promising learning procedures for deep neural networks**.
    - Backpropagation
    - **Boltzmann Machines** which performed **unsupervised contrastive learning**.



참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

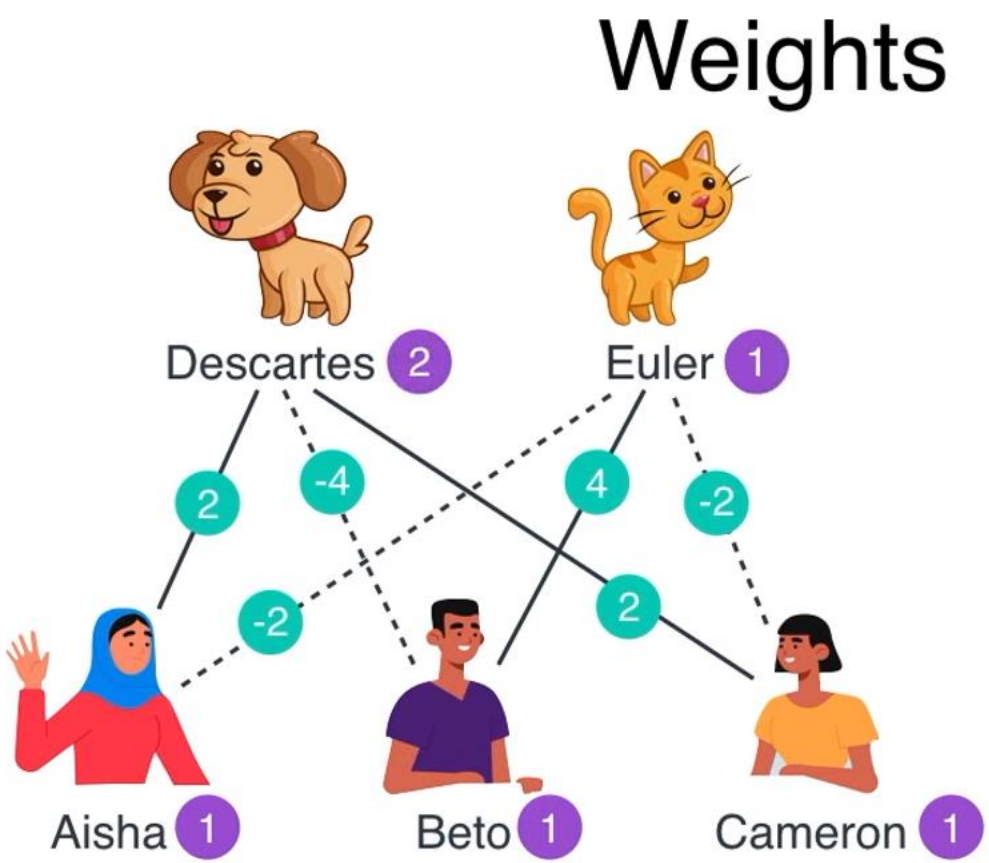
Part 2,

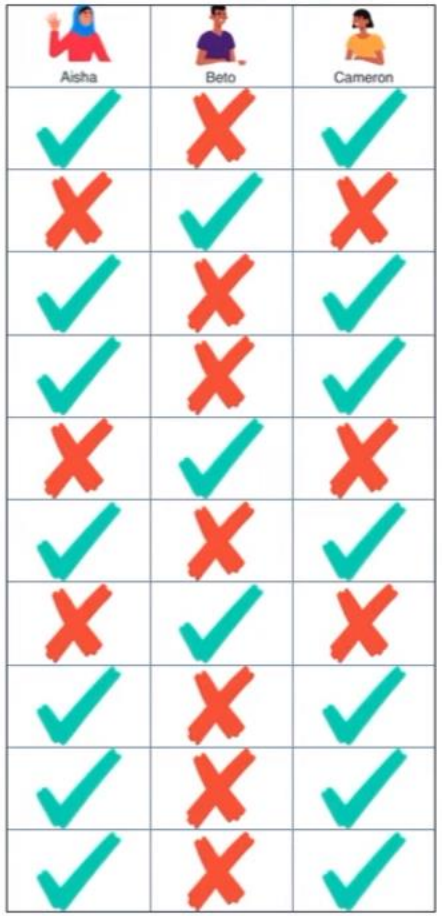
How FF relates to other contrastive learning techniques



참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

How FF relates to other contrastive learning techniques

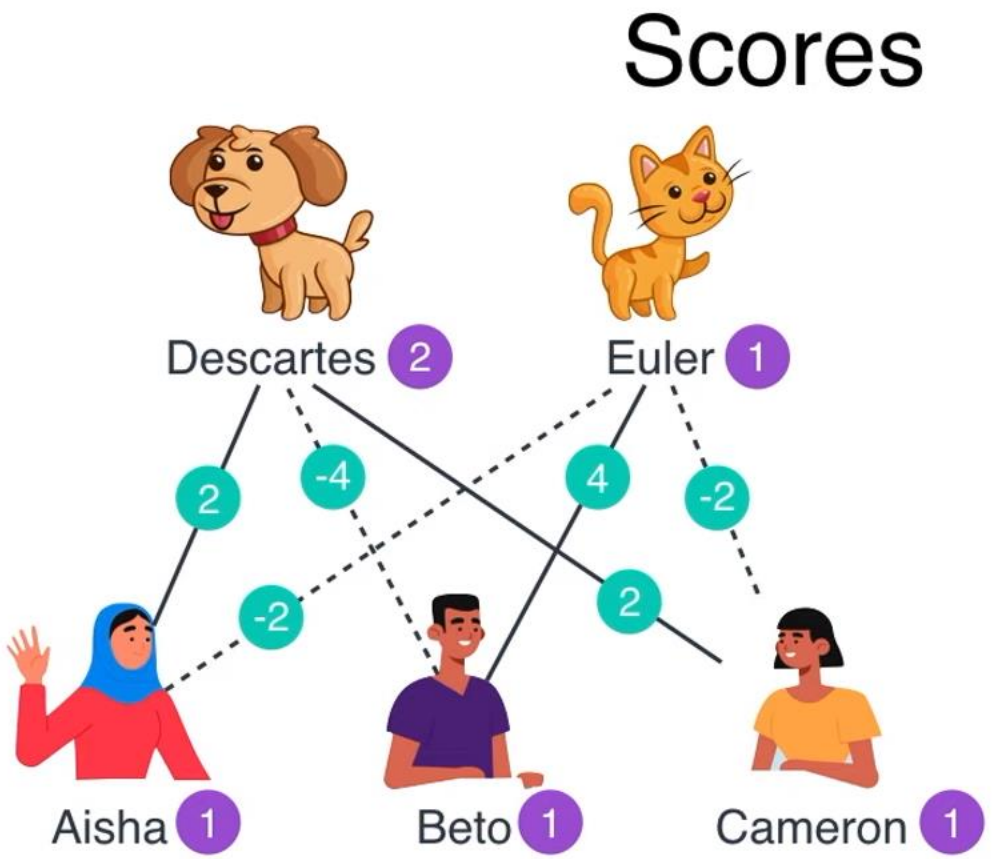


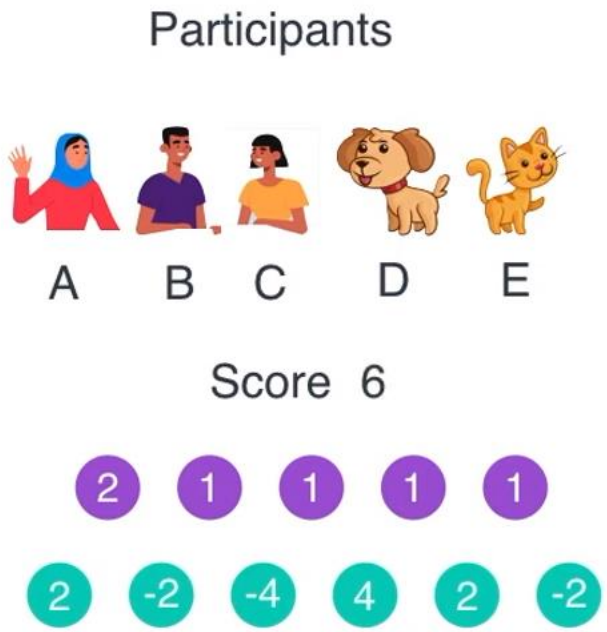


참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

Part 2,

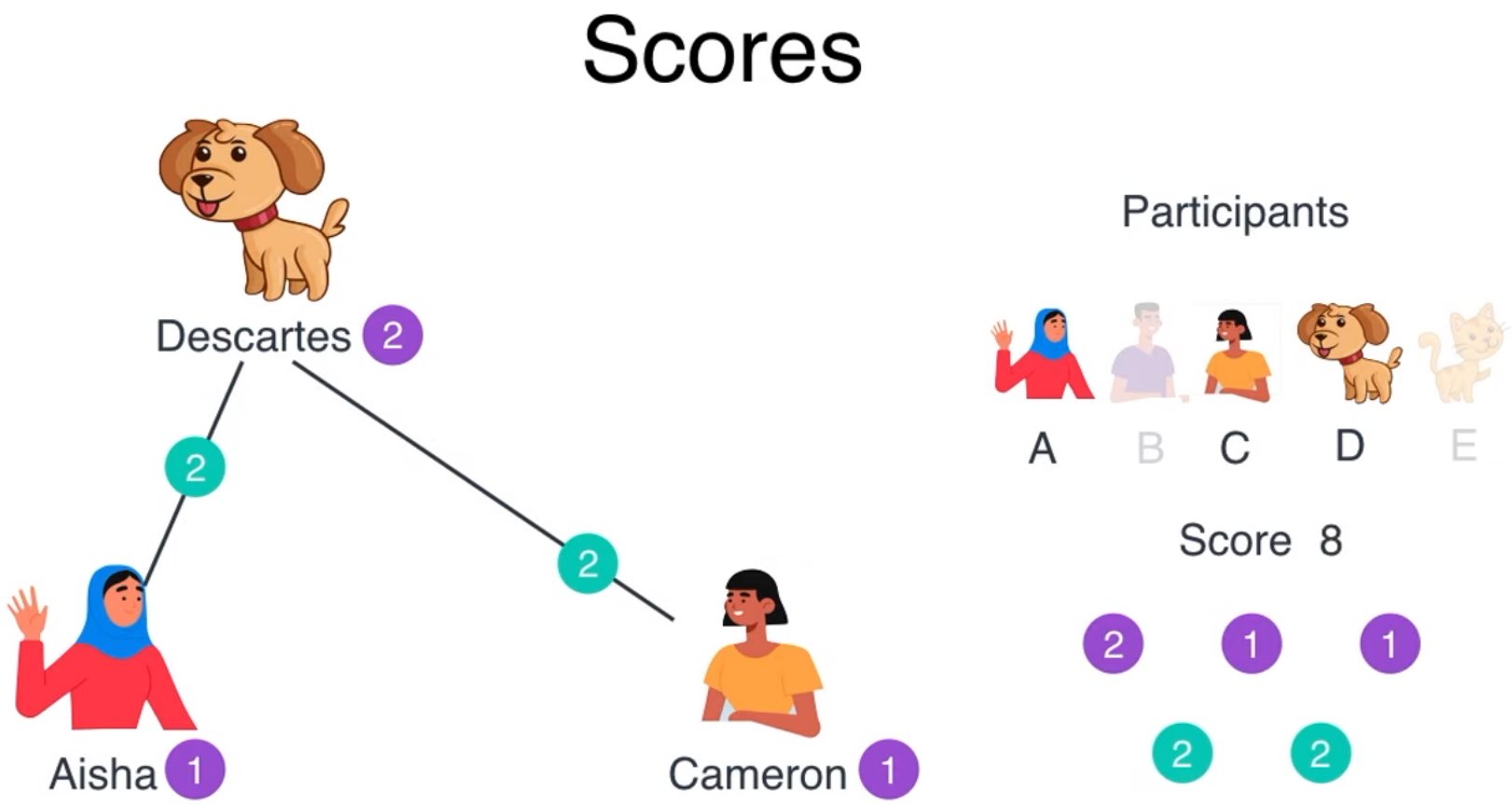
How FF relates to other contrastive learning techniques





참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

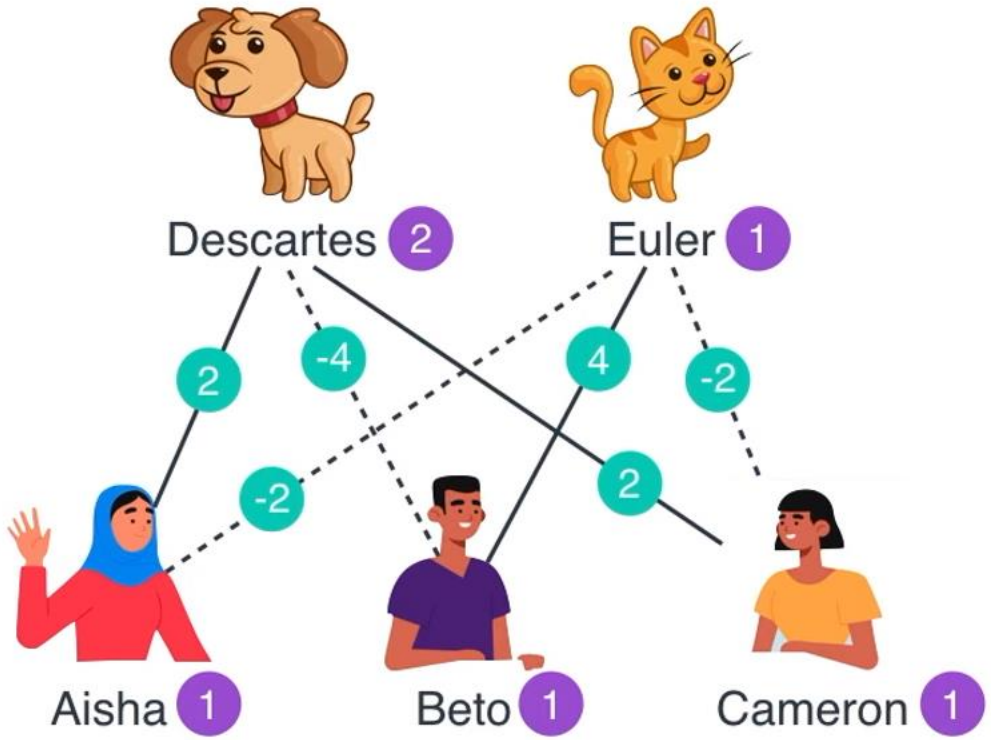
How FF relates to other contrastive learning techniques



참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

Part 2,

How FF relates to other contrastive learning techniques

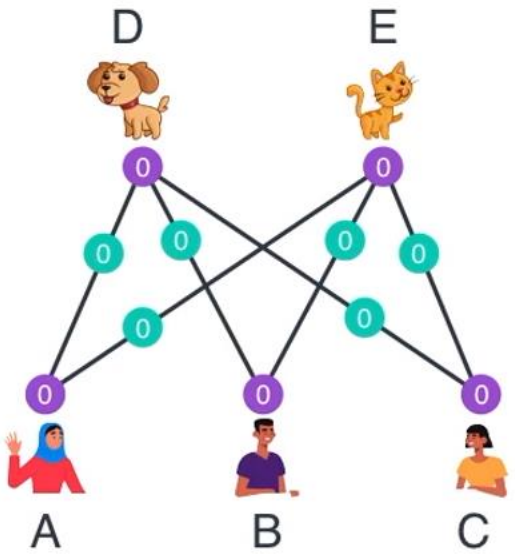


Scenario	Score	e^{Score}	Probability
None	0	1	0
A	1	2.72	0
B	1	2.72	0
C	1	2.72	0
D	2	7.38	0
E	1	2.72	0
AB	2	7.38	0
AC	2	7.38	0
AD	5	148.41	0.02
AE	0	2.72	0
BC	2	7.38	0
BD	-2	0.14	0
BE	7	1096.63	0.17
CD	5	148.41	0.02
CE	0	1	0
DE	3	20.08	0
ABC	3	20.08	0
ABD	1	2.72	0
ABE	6	403.43	0.06
ACD	8	2980.96	0.45
ACE	-1	0.37	0
ADE	4	54.6	0
BCD	1	2.72	0
BCE	6	403.43	0.06
BDE	4	54.6	0
CDE	4	54.6	0
ABCD	4	54.6	0.02
ABCE	5	148.41	0.02
ABDE	5	148.41	0.02
ACDE	5	148.41	0.02
BCDE	5	148.41	0.02
ABCDE	6	403.43	0.06

참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

Part 2,

How FF relates to other contrastive learning techniques



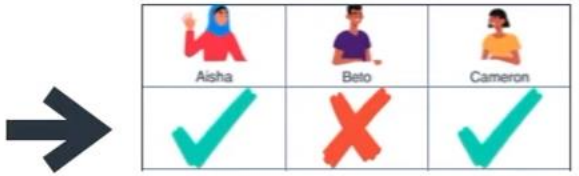
Scenario	Score	e^{Score}	Probability
None	0	1	1/32
A	0	1	1/32
B	0	1	1/32
C	0	1	1/32
D	0	1	1/32
E	0	1	1/32
AB	0	1	1/32
AC	0	1	1/32
AD	0	1	1/32
AE	0	1	1/32
BC	0	1	1/32
BD	0	1	1/32
BE	0	1	1/32
CD	0	1	1/32
CE	0	1	1/32
DE	0	1	1/32

Scenario	Score	e^{Score}	Probability
ABC	0	1	1/32
ABD	0	1	1/32
ABE	0	1	1/32
ACD	0	1	1/32
ACE	0	1	1/32
ADE	0	1	1/32
BCD	0	1	1/32
BCE	0	1	1/32
BDE	0	1	1/32
CDE	0	1	1/32
ABCD	0	1	1/32
ABCE	0	1	1/32
ABDE	0	1	1/32
ACDE	0	1	1/32
BCDE	0	1	1/32
ABCDE	0	1	1/32

참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

Part 2,

How FF relates to other contrastive learning techniques



A, C, and no B

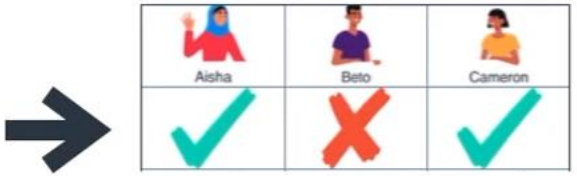
None
A
B
C
D
E
AB
↑ AC
AD
AE
BC
BD
BE
CD
CE
DE

ABC
ABD
ABE
↑ ACD
↑ ACE
ADE
BCD
BCE
BDE
CDE
ABCD
ABCE
ABDE
↑ ACDE
BCDE
ABCDE

참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

Part 2,

How FF relates to other contrastive learning techniques



A, C, and no B

All scenarios

None
A
B
C
D
E
AB
AC
AD
AE
BC
BD
BE
CD
CE
DE

ABC
ABD
ABE
ACD
ACE
ADE
BCD
BCE
BDE
CDE
ABCD
ABCE
ABDE
ACDE
BCDE
ABCDE

참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

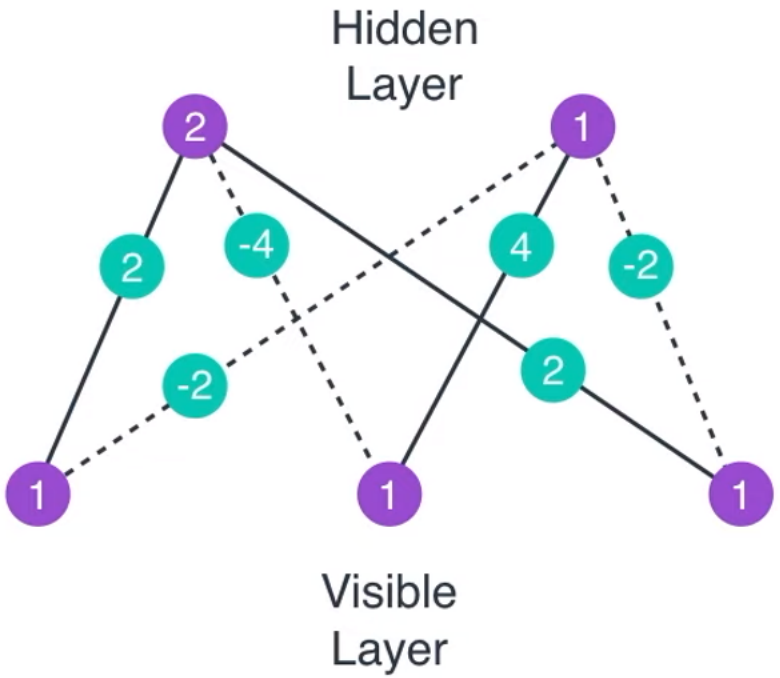
Part 2,

How FF relates to other contrastive learning techniques

참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

How FF relates to other contrastive learning techniques

Maximizing the probability of the data



Find $\arg\max_W \prod_{v \in V} P(v)$

Maximize $\arg\max_W \mathbb{E}[\log P(v)]$

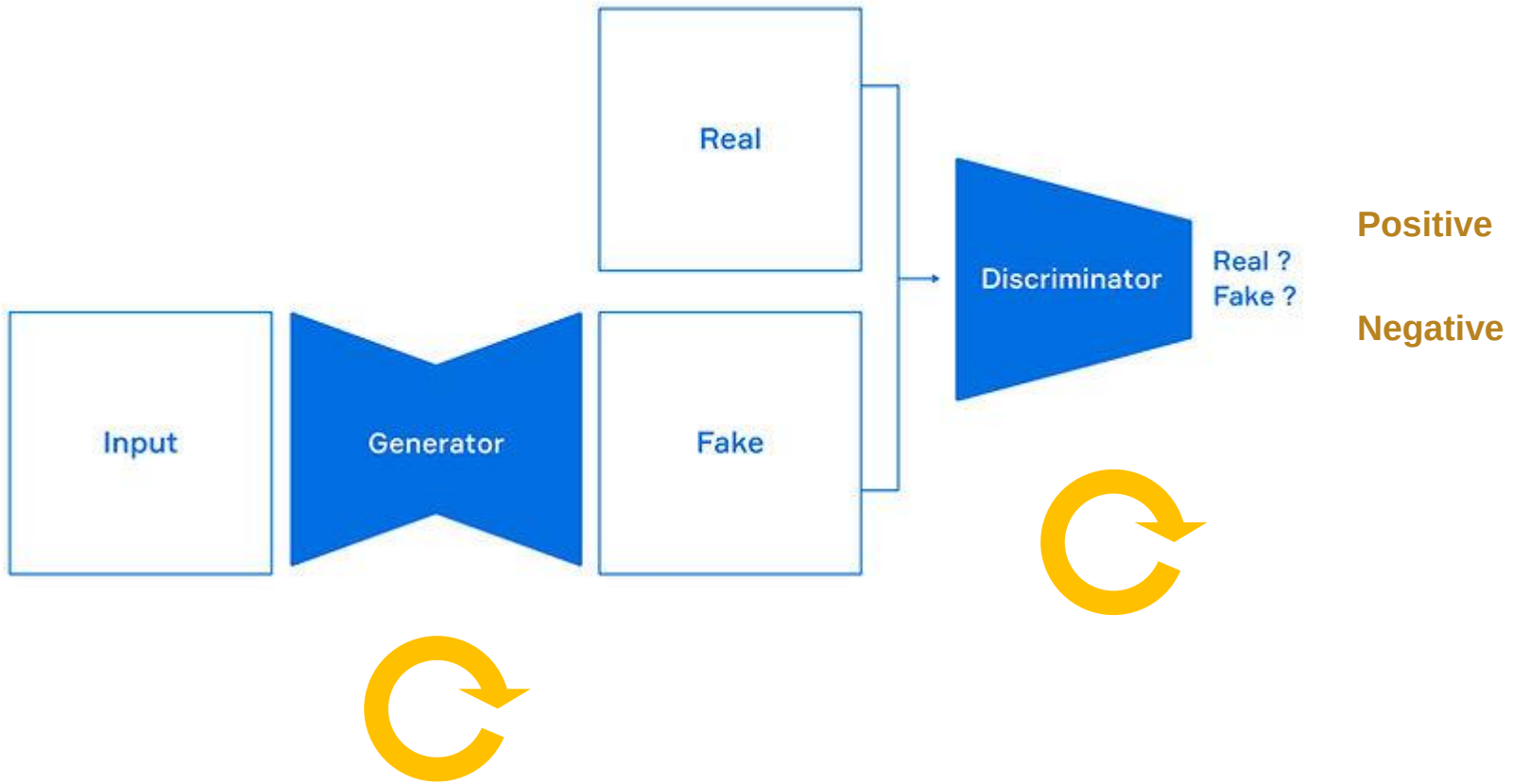
Derivative: $\frac{\partial}{\partial W} \log P(v_n)$

$$= \mathbb{E}\left[\frac{\partial}{\partial W} - E(v,h) \mid v = v_n\right] - \mathbb{E}\left[\frac{\partial}{\partial W} - E(v,h)\right]$$

참조:https://www.youtube.com/watch?v=Fkw0_aAtwIw

How FF relates to other contrastive learning techniques

- **Relationship to Generative Adversarial Networks**



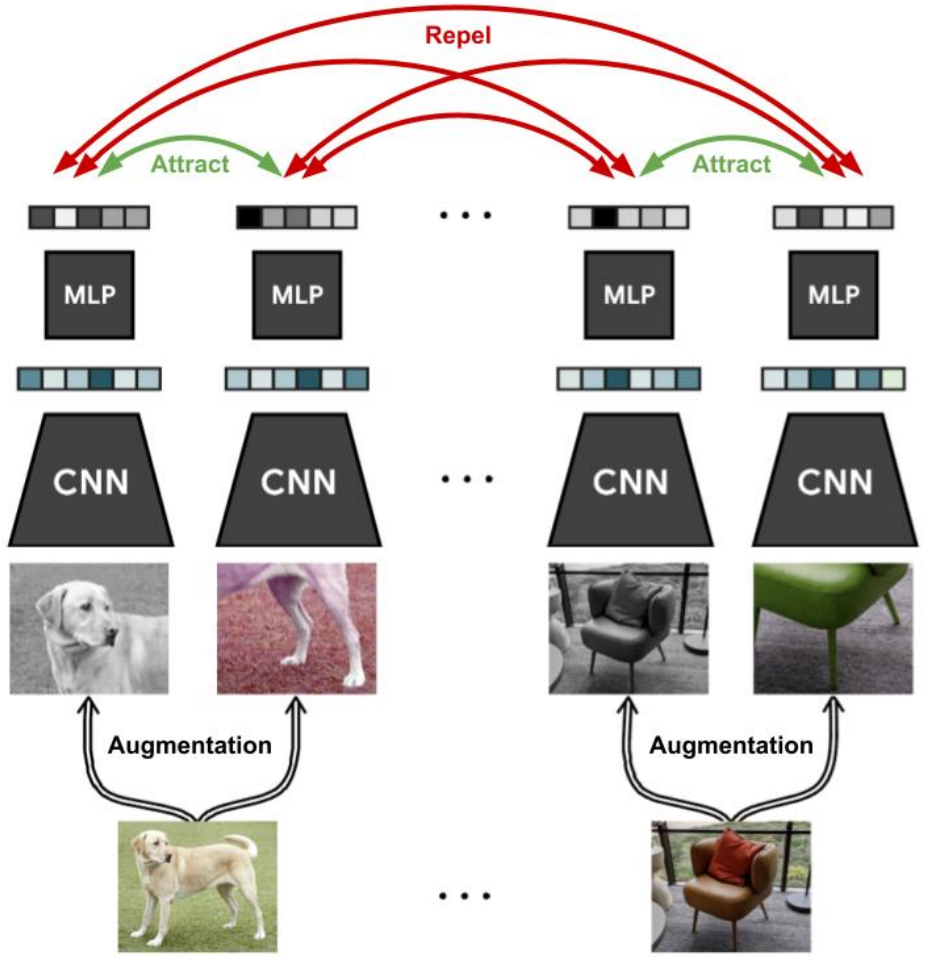
참조:https://www.paimedialab.com/post/gans-%EC%96%B4%EB%96%BB%EA%B2%8C-%ED%99%9C%EC%9A%A9%EB%90%98%EA%B3%A0-%EC%9E%88%EC%9D%84%EA%B9%8C

How FF relates to other contrastive learning techniques

- **Relationship to contrastive methods that compare representations of two different image crops**

Positive

Negative



참조:https://uvadlc-notebooks.readthedocs.io/en/latest/tutorial_notebooks/JAX/tutorial17/SimCLR.html

How FF relates to other contrastive learning techniques

- **Noise Contrastive Estimation**

Data

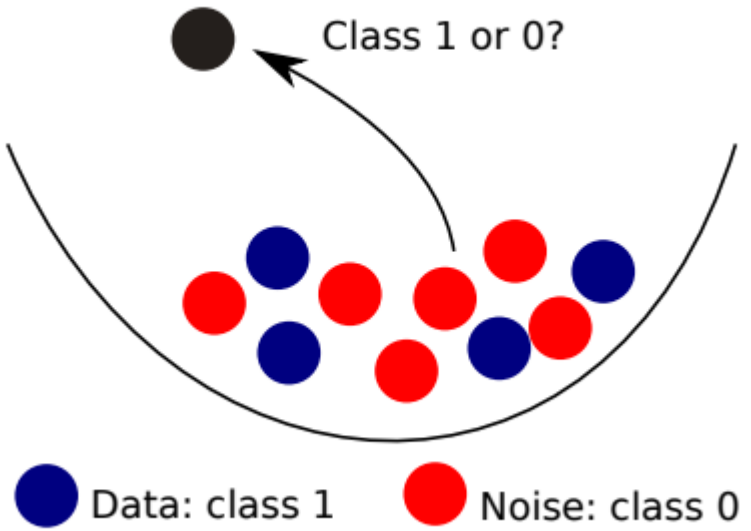
$$X = \{x_1, x_2, \dots, x_{T_d}\}$$

$$X \sim P_d \qquad \textit{(unknow PDF)}$$

Noise

$$Y = \{y_1, y_2, \dots, y_{T_n}\}$$

$$X \sim P_n \qquad \textit{(know PDF)}$$



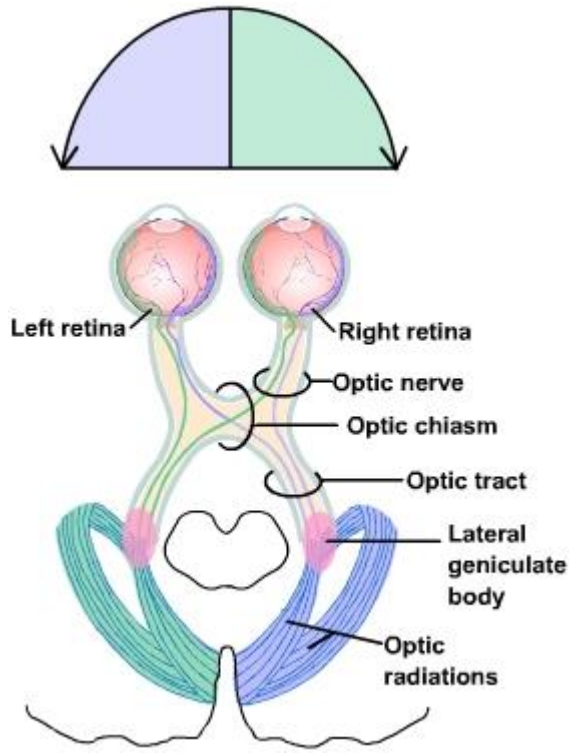
참조: http://incredible.ai/nlp/2017/11/25/Word2Vec-Noise-Constrastive-Estimation/

Part 3,

What is wrong with backpropagation

What is wrong with backpropagation

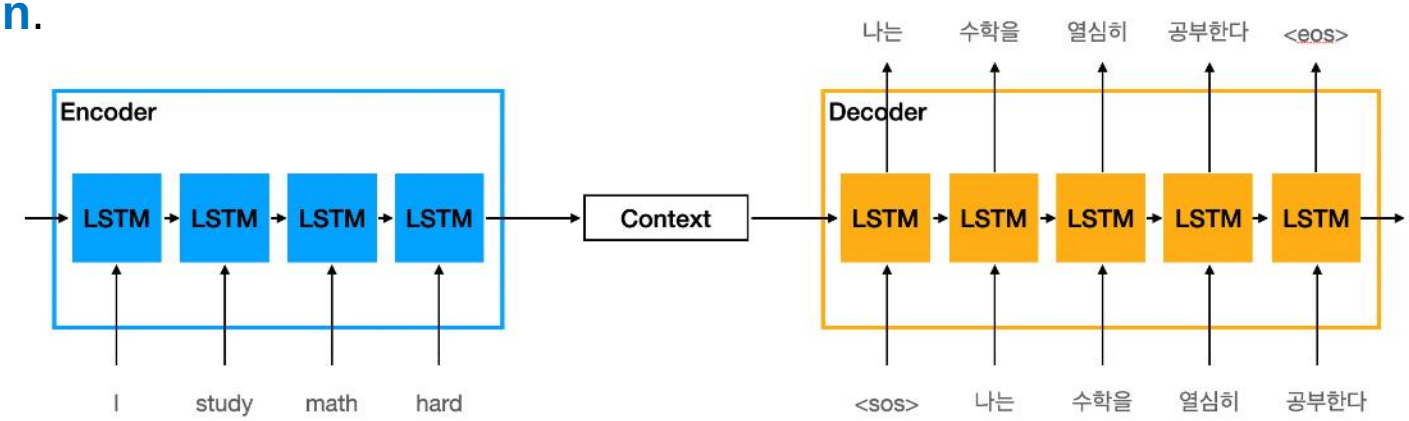
- **Backpropagation 학습 방법과 대뇌피질(cortex) 학습 방법은 다르다**
  - There is no convincing evidence that cortex explicitly **propagates error derivatives** or **stores neural activities** for use in a subsequent backward pass.
  - The top-down connections from one cortical area to an area that is earlier in the **visual pathway do not mirror the bottom-up connections** as would be expected if backpropagation was being used in the visual system.



그림참조: https://nba.uth.tmc.edu/neuroscience/s2/chapter15.html

What is wrong with backpropagation

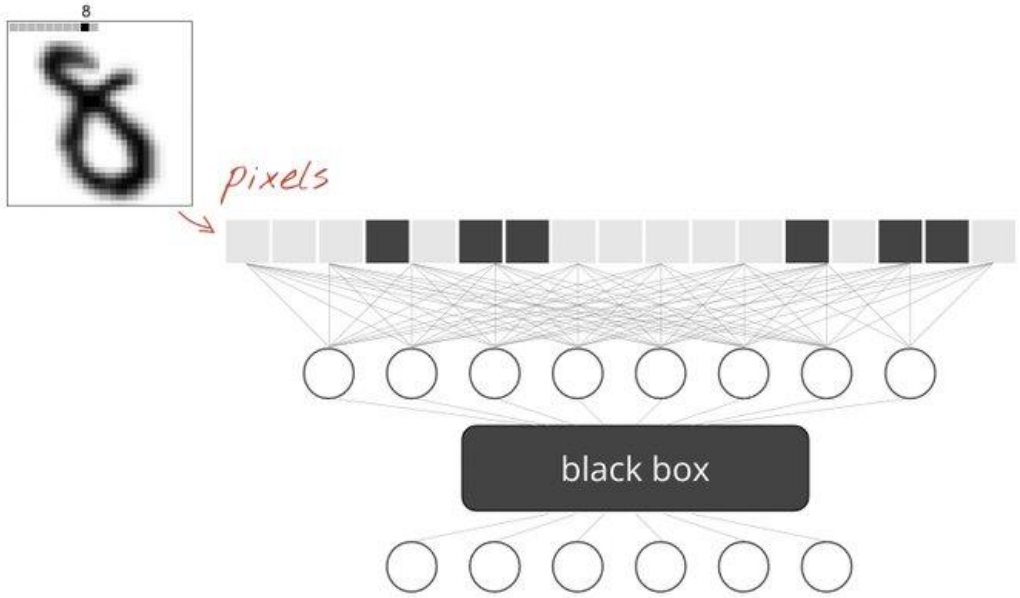
- **시퀀스를 학습하는 방법으로 시간을 통한 backpropagation은 특히 불가능**
  - To deal with the **stream of sensory input** without taking frequent time-outs, the brain needs to pipeline sensory data through different stages of sensory processing and it needs a learning procedure that can **learn on the fly**.
  - The representations in later stages of the pipeline may provide top-down information that influences the representations in earlier stages of the pipeline at a later time step, but the perceptual system needs to perform inference and **learning in real time without stopping to perform backpropagation**.



그림참조: https://velog.io/@nkw011/seq-to-seq

What is wrong with backpropagation

- **정확한 미분 계산을 위한 Forward 패스에서 수행되는 계산에 대한 완벽한 지식이 필요**
  - If we **insert a black box** into the forward pass, it is no longer possible to perform backpropagation unless we learn a differentiable model of the black box.
  - As we shall see, the black box does not change the learning procedure at all for the **Forward-Forward Algorithm because there is no need to backpropagate** through it.



What is wrong with backpropagation

- **FF 알고리즘 장단점**
  - FF는 forward computation의 **정확한 세부 사항을 알 수 없을 때**도 사용가능
  - Pipelining sequential data를 **activity를 저장하거나 오류를 전파하기 위해 멈추지 않고** 학습가능
  - FF는 backpropagation에 비해서 다소 **느리고** 몇 가지 toy 문제에 대해 **일반화가 잘 되지 않음**
  - **전력이 문제가 되지 않는 애플리케이션**에 대한 backpropagation을 대체할 가능성은 낮음
  - FF 알고리즘이 우수할 수 있는 두 가지 영역
    - a model of **learning in cortex**
    - a way of making use of **very low-power analog hardware**

Part 4,

The Forward-Forward Algorithm

The Forward-Forward Algorithm

- The Forward-Forward algorithm
  - **Greedy multi-layer learning procedure** inspired by **Boltzmann machines** (Hinton and Sejnowski, 1986)
  - **Noise Contrastive Estimation** (Gutmann and Hyvärinen, 2010).

The Forward-Forward Algorithm

- The idea is to replace the forward and backward passes of backpropagation by **two forward passes** that operate in exactly the same way as each other, but on **different data and with opposite objectives**.
  - The **positive pass** operates on **real data** and adjusts the weights to **increase the goodness in every hidden layer**.
  - The **negative pass** operates on "**negative data**" and adjusts the weights to decrease the goodness in every hidden layer.
- This paper explores two different measures of **goodness** - **the sum of the squared neural activities** and **the negative sum of the squared activities**, but many other measures are possible.

$$p(positive) = \sigma\left(\sum_j y_j^2 - \theta\right)$$

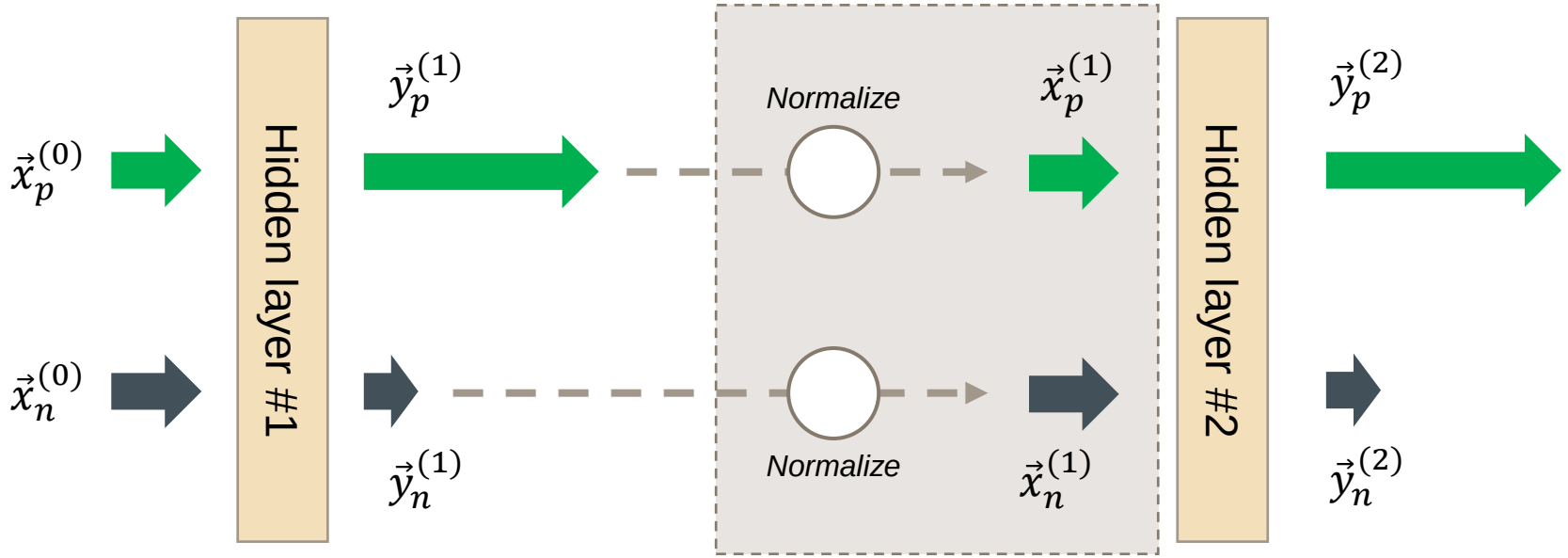
The Forward-Forward Algorithm

- Let us **suppose** that the **goodness function** for a layer is simply the **sum of the squares of the activities** of the rectified linear neurons in that layer.
  - The aim of the learning is to make the goodness be well **above some threshold value for real data** and well **below that value for negative data**.
  - The **negative data** may be **predicted by the neural net** using top-down connections, or it may be **supplied externally**.

$$p(positive) = \sigma\left(\sum_j y_j^2 - \theta\right)$$

The Forward-Forward Algorithm

- FF **normalizes the length of the hidden vector** before using it **as input to the next**.
  - The **length is used to define the goodness** for that layer and only the **orientation is passed to the next layer**.



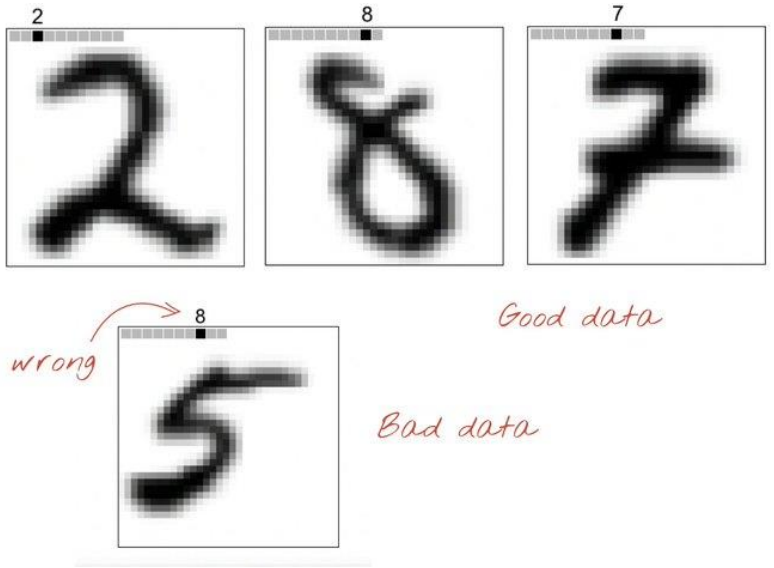
Part 4,

The Forward-Forward Algorithm

```python
class Layer(nn.Linear):
    def __init__(self, in_features, out_features,
                 bias=True, device=None, dtype=None):
        super().__init__(in_features, out_features, bias, device, dtype)
        self.relu = torch.nn.ReLU()
        self.opt = Adam(self.parameters(), lr=0.03)
        self.threshold = 2.0
        self.num_epochs = 1000

    def forward(self, x):
        # normalize previous layer input
        x_direction = x / (x.norm(2, 1, keepdim=True) + 1e-4)
        return self.relu(
            torch.mm(x_direction, self.weight.T) +
            self.bias.unsqueeze(0))

    def train(self, x_pos, x_neg):
        for i in tqdm(range(self.num_epochs)):
            g_pos = self.forward(x_pos).pow(2).mean(1)
            g_neg = self.forward(x_neg).pow(2).mean(1)
            # The following loss pushes pos (neg) samples to
            # values larger (smaller) than the self.threshold.
            loss = torch.log(1 + torch.exp(torch.cat([
                -g_pos + self.threshold,
                g_neg - self.threshold]))).mean()
            self.opt.zero_grad()
            # this backward just compute the derivative and hence
            # is not considered backpropagation.
            loss.backward()
            self.opt.step()
        return self.forward(x_pos).detach(), self.forward(x_neg).detach()
```



$$p(positive) = \sigma\left(\sum_j y_j^2 - \theta\right)$$

Pytorch: https://github.com/mohammadpz/pytorch_forward_forward
Tensorflow: https://github.com/amirrezarajabi/Tensorflow-Forward-Forward


Q&A

Part 5,

Some experiments with FF

Some experiments with FF

- **The backpropagation baseline**
  - Dataset: **NMIST**
  - A few **fully connected** hidden layers (ReLU)
  - **No regularizers** (dropout)
  - **20 epochs**
  - **1.4% test error**

Some experiments with FF

- **A simple unsupervised example of FF**
  - First, if we have a **good source of negative data**, does it **learn effective multi-layer representations** that capture the structure in the data?
    - Random mask 및 1–mask 생성
    - hybrid images for the negative data 생성
  - NN: **four fully connected layers** (ReLUs)
    - 100 epochs
    - softmax
    - **1.37%** test error
  - local receptive fields (without weight-sharing)
    - 60 epochs
    - **1.16%** test error

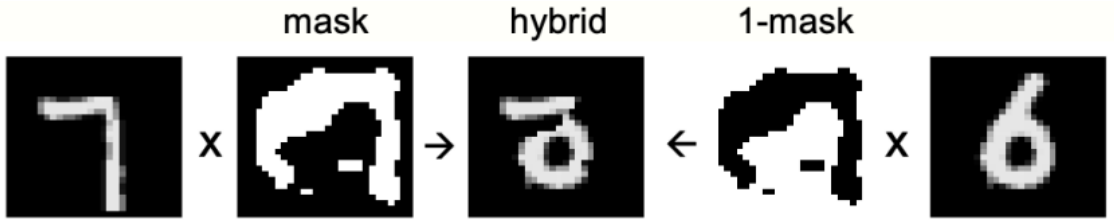
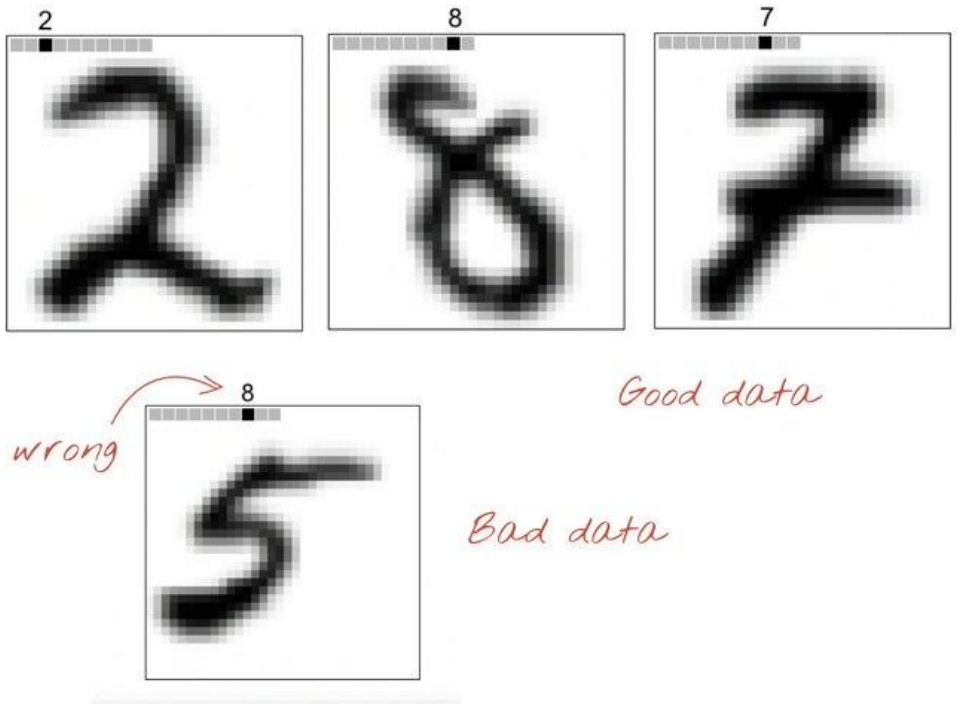


Figure 1: A hybrid image used as negative data

Some experiments with FF

- **A simple supervised example of FF**
  - The **positive data** consists of an image with the **correct label** and the **negative data** consists of an image with the **incorrect label**.
  - Inference
    - **Softmax**
    - Infer for each label separately and choice the **highest accumulated goodness**
  - **4 hidden layers** (ReLU)
    - 60 epoch / **1.36%** test errors
  - Doubling the learning rate
    - 40 epoch / **1.46%** test errors



Some experiments with FF

- **A simple supervised example of FF** **(이미지 처리팀 최승준님 테스트)**
  - train batch = 60000
    - test error: 0.06850004196166992
  - train batch = 1000
    - test error: 0.0755000114440918
  - train batch = 100
    - test error: 0.9020000025629997

  - Batch size 크기가 **줄어들수록 오류가 늘어나는 현상**이 발생 함

Some experiments with FF

- **A simple supervised example of FF**
  - We can **augment the training data by jittering** the images by up to two pixels in each direction to get **25 different shifts** for each image.
  - 500 epochs / 0.64% test error similar to a CNN (backpropagation)
  - We also get interesting **receptive fields** in the first hidden layer.

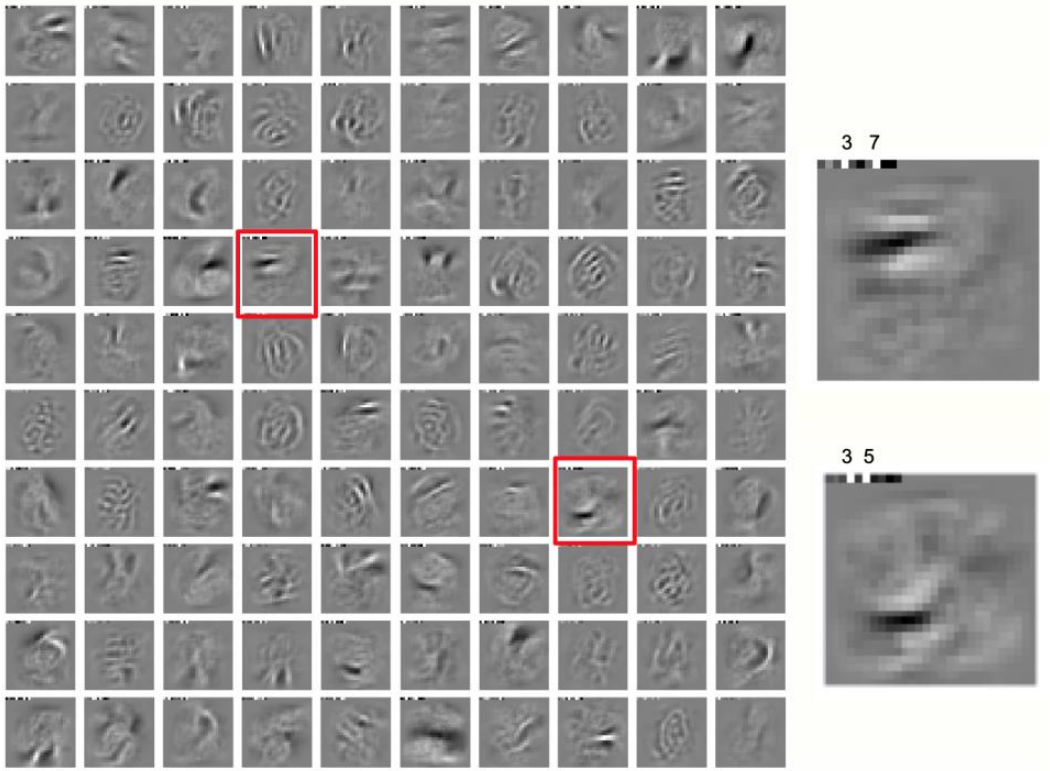


Figure 2: The receptive fields of 100 neurons in the first hidden layer of the network trained on jittered MNIST. The class labels are represented in the first 10 pixels of each image.

Some experiments with FF

- **Using FF to model top-down effects in perception**
  - FF's learned in later layers **cannot affect what is learned in earlier layers**. (seems like a major weakness)
  - the activity vector at each layer is determined by the normalized activity vectors at both the **layer above** and the **layer below** at the **previous time-step**.
  - **0.3 of the previous** pre-normalized state plus **0.7 of the computed new state**.
  - **8 synchronous iterations** and picking the label that has the highest goodness averaged over **iterations 3 to 5**.
  - 60 epochs / **1.31%** test error.

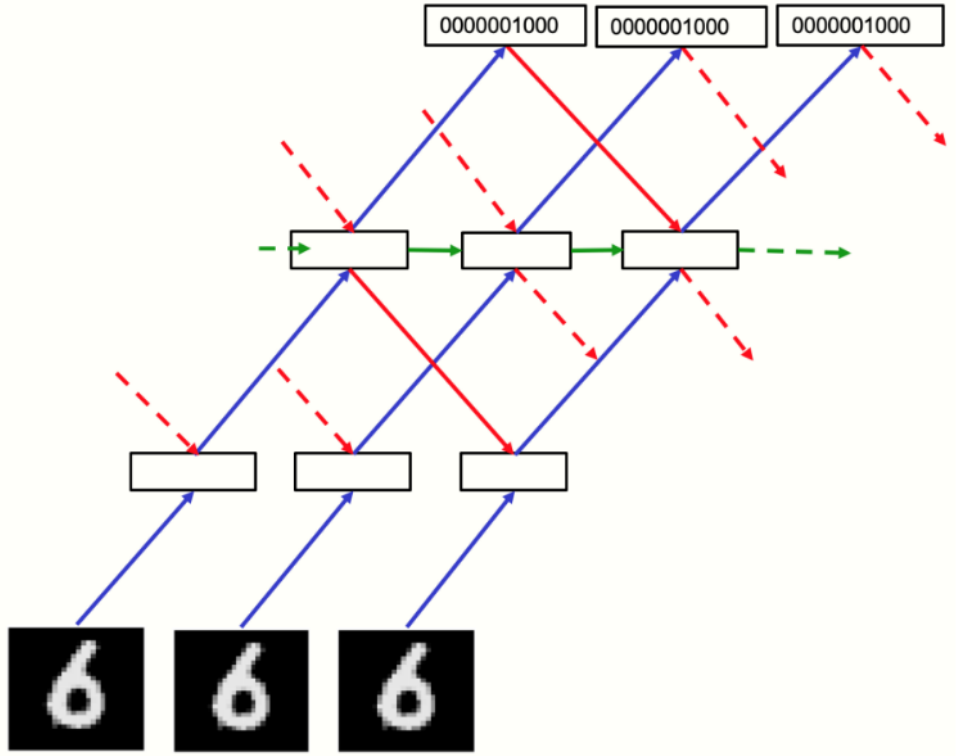


Figure 3: The recurrent network used to process video.

Experiments with CIFAR-10

- **Experiments with CIFAR-10**
  - **FF is comparable in performance** to backpropagation for images that contain **highly variable backgrounds**.
  - **Two or three hidden layers** (ReLUs).
  - FF run for **10 iterations** / accumulate over **iterations 4 to 6**.
  - **FF is worse** than backpropagation slightly, even when there are **complicated confounding backgrounds**.
  - **Gap** between the two procedures does **not increase** with **more hidden layers**.

learning procedure	testing procedure	number of hidden layers	training % error rate	test % error rate
BP		2	0	37
FF min ssq	compute goodness for every label	2	20	41
FF min ssq	one-pass softmax	2	31	45
FF max ssq	compute goodness for every label	2	25	44
FF max ssq	one-pass softmax	2	33	46
BP		3	2	39
FF min ssq	compute goodness for every label	3	24	41
FF min ssq	one-pass softmax	3	32	44
FF max ssq	compute goodness for every label	3	21	44
FF max ssq	one-pass softmax	3	31	46

Table 1: A comparison of backpropagation and FF on CIFAR-10 using non-convolutional nets with local receptive fields of size 11x11 and 2 or 3 hidden layers. One version of FF is trained to maximize the sum of the squared activities on positive cases. The other version is trained to minimize it and this gives slightly better test performance. After training with FF, a single forward pass through the net is a quick but sub-optimal way to classify an image. It is better to run the net with a particular label as the top-level input and record the average goodness over the middle iterations. After doing this for each label separately, the label with the highest goodness is chosen. For a large number of labels, a single forward pass could be used to get a candidate list of which labels to evaluate more thoroughly.

Part 6,

Learning fast and slow

Learning fast and slow

- $\Delta w_j = 2 \in \frac{\partial \log(p)}{\partial \sum_j y_j^2} y_j x$
- Change in the activity of neuron: $\Delta \boldsymbol{w_j x}$
  - The **only term that depends on j in the change of activity caused by the weight update is y_i**, so all the hidden activities change by the same proportion and the weight update **does not change the orientation** of the activity vector.
- The fact that the **weight update does not change the layer normalized output** for that input vector means that it is **possible to perform simultaneous online weight updates in many different layers**.
- The **learning rate** that achieves this is given by:

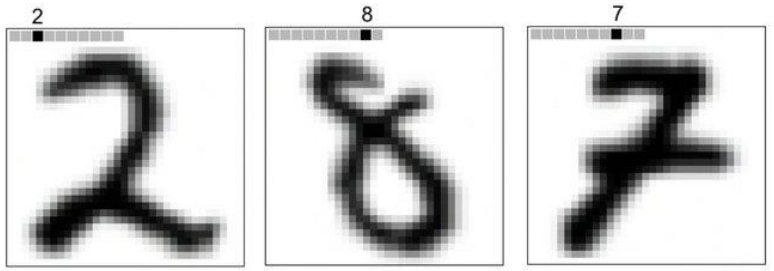
$$\in = \sqrt{\frac{S^*}{S_L} - 1}$$

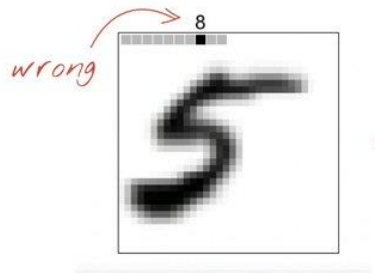
Learning fast and slow

```python
class Layer(nn.Linear):
    def __init__(self, in_features, out_features,
                 bias=True, device=None, dtype=None):
        super().__init__(in_features, out_features, bias, device, dtype)
        self.relu = torch.nn.ReLU()
        self.opt = Adam(self.parameters(), lr=0.03)
        self.threshold = 2.0
        self.num_epochs = 1000

    def forward(self, x):
        # normalize previous layer input
        x_direction = x / (x.norm(2, 1, keepdim=True) + 1e-4)
        return self.relu(
            torch.mm(x_direction, self.weight.T) +
            self.bias.unsqueeze(0))

    def train(self, x_pos, x_neg):
        for i in tqdm(range(self.num_epochs)):
            g_pos = self.forward(x_pos).pow(2).mean(1)
            g_neg = self.forward(x_neg).pow(2).mean(1)
            # The following loss pushes pos (neg) samples to
            # values larger (smaller) than the self.threshold.
            loss = torch.log(1 + torch.exp(torch.cat([
                -g_pos + self.threshold,
                g_neg - self.threshold]))).mean()
            self.opt.zero_grad()
            # this backward just compute the derivative and hence
            # is not considered backpropagation.
            loss.backward()
            self.opt.step()
        return self.forward(x_pos).detach(), self.forward(x_neg).detach()
```





Good data

Bad data

$$p(positive) = \sigma\left(\sum_j y_j^2 - \theta\right)$$

Pytorch: https://github.com/mohammadpz/pytorch_forward_forward
Tensorflow: https://github.com/amirrezarajabi/Tensorflow-Forward-Forward

Part 7,

Mortal Computation

Mortal Computation

- **The relevance of FF to analog hardware**
  - An energy efficient way to multiply an activity vector by a weight matrix is to implement **activities as voltages** and **weights as conductances**.
  - Unfortunately, it is difficult to implement the backpropagation procedure in an equally efficient way, so people have resorted to **using A-to-D converters and digital computations for computing gradients**.
  - FF should make these **A-to-D converters unnecessary**.

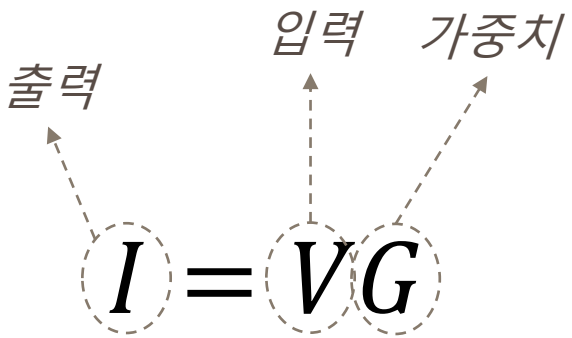
Mortal Computation

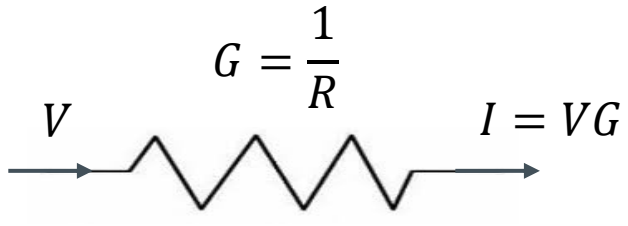
- **The relevance of FF to analog hardware**

$$V = IR$$

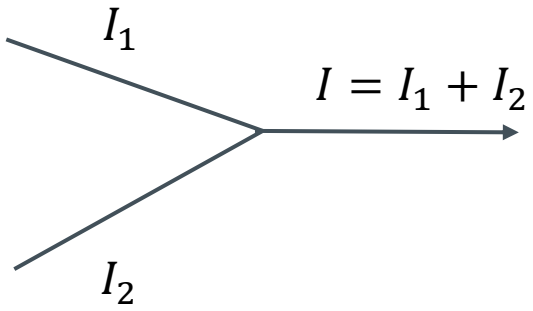
$$I = V\frac{1}{R}$$

$$where\ \ G = \frac{1}{R}$$





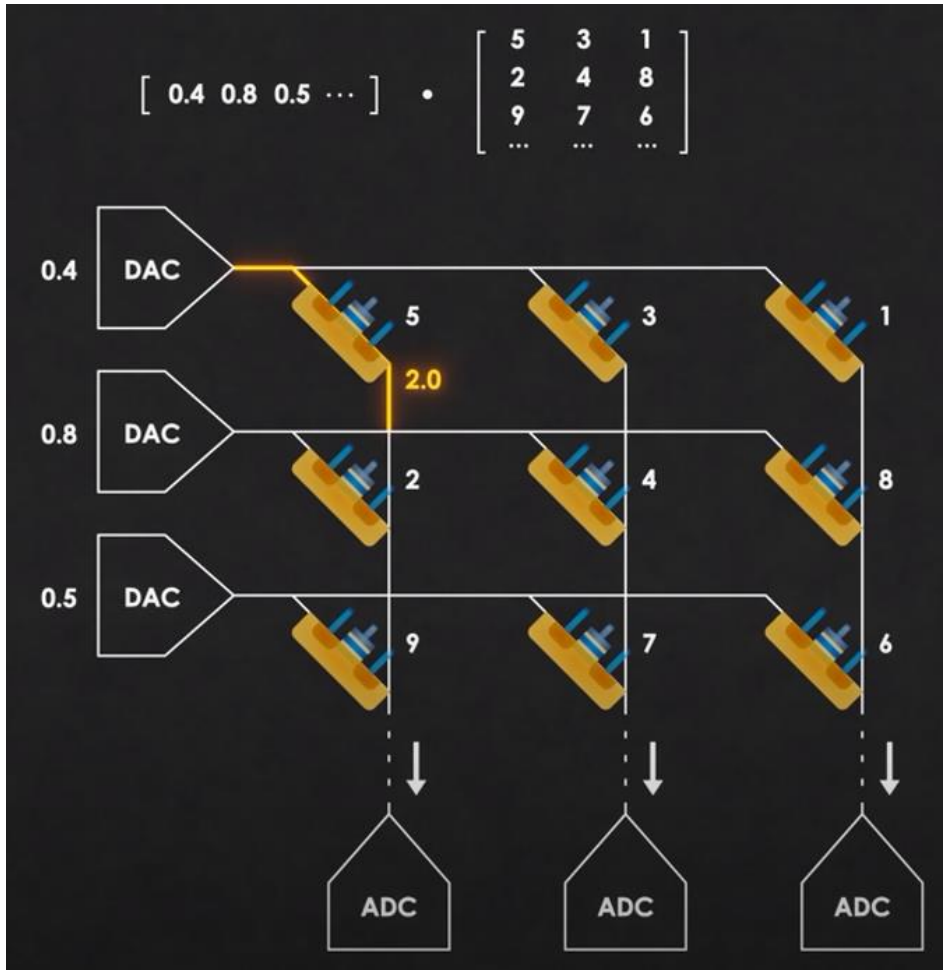
곱셈 계산방법



덧셈 계산방법

Mortal Computation

- **The relevance of FF to analog hardware**



참고: https://www.youtube.com/watch?v=GVsUOuSjvcg&t=940s

Mortal Computation

- **Immortal**: The **knowledge does not die** when the **hardware dies**.
  - The **software should be separable from the hardware** so that the same program or the same set of weights **can be run on a different physical copy of the hardware**.
- **Mortal**: It should be possible to achieve huge **savings in the energy** required to perform a computation and in the **cost of fabricating the hardware** that executes the computation.
  - These **parameter values are only useful for that specific hardware instance**, so the computation they perform is mortal: **it dies with the hardware**.
- The function itself can be **transferred (approximately) to a different piece of hardware** by using **distillation**.

참고: https://www.youtube.com/watch?v=sghvwkXV3VU

Part 8,

Future work

Future work

- FF produce a **generative model** of images or video?
- What is the **best goodness function** to use?
- What is the **best activation function** to use?
- For spatial data, can FF **benefit from having lots of local goodness functions** for different regions of the image?
- For sequential data, is it possible to use **fast weights to mimic a simplified transformer**?
- Can FF **benefit** from having a set of feature detectors that try to **maximize their squared activity** and a set of constraint violation detectors that try to **minimize their squared activity**?


Q&A